

Entornos virtuales con Conda

Edison Achalma

Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Palabras Claves: keyword1, keyword2

Tabla de contenidos

Introduction	2	6.1	Uso de conda install	5
1 Introducción	2	6.2	Uso de pip install	5
2 ¿Qué es un entorno virtual?	2	6.3	Ver información del paquete de entorno:	6
2.1 ¿Cuáles son los beneficios de trabajar con entornos virtuales en proyectos de Python en Linux?	2	6.4	Importar y exportar entornos: . . .	6
3 Anaconda	3	7 Coclusión		6
4 Instalación de Anaconda en Ubuntu Linux	3	8 Publicaciones Similares		6
5 Configuración de un entorno virtual en Conda	4			
6 Instalación de paquetes y bibliotecas en un entorno virtual	5			

 Edison Achalma

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; <https://credit.niso.org/>) de la siguiente manera: Edison Achalma: conceptualización, redacción

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Email: elmer.achalma.09@unsch.edu.pe

Entornos virtuales con Conda

1 Introducción

¡Bienvenidos al blog! En esta ocasión, exploraremos un tema fundamental para cualquier desarrollador o entusiasta de Python: la configuración de un entorno virtual utilizando Anaconda. Si alguna vez te has preguntado cómo mantener tus proyectos de Python aislados, organizados y libres de conflictos de dependencias, estás en el lugar correcto.

¿Alguna vez has encontrado problemas al trabajar en múltiples proyectos de Python, donde las diferentes versiones de las bibliotecas y paquetes interfieren entre sí? ¡No te preocupes! Configurar un entorno virtual es la solución perfecta para mantener todo bajo control.

Imagina tener la capacidad de crear espacios de trabajo aislados y personalizados para cada proyecto, sin preocuparte por conflictos entre las versiones de las bibliotecas o paquetes. Con Anaconda, una poderosa plataforma de gestión de paquetes y ambientes virtuales, puedes lograr precisamente eso.

En este blog, te guiaremos paso a paso en la configuración de tu primer entorno virtual utilizando Anaconda. Aprenderás cómo crear ambientes virtuales, instalar paquetes y bibliotecas específicos, y alternar fácilmente entre ellos para cada proyecto.

Ya sea que seas un desarrollador principiante o experimentado, este blog te brindará los conocimientos necesarios para dominar el arte de la configuración de entornos virtuales con Anaconda. ¡Prepárate para simplificar tu vida como desarrollador de Python y llevar tus proyectos al siguiente nivel!

¡Comencemos esta emocionante aventura de configuración de entornos virtuales con Anaconda!

2 ¿Qué es un entorno virtual?

Si estas en el mundo de la programación en Python, es posible que hayas oído hablar del término “entorno virtual”.

Cuando hablamos de un entorno virtual, nos referimos a un ambiente aislado y autónomo donde puedes desarrollar y ejecutar tus proyectos de

Python de manera independiente. En otras palabras, es como tener una burbuja protegiendo tus proyectos y asegurándote de que las bibliotecas y paquetes que utilices no entren en conflicto con otras versiones o dependencias de Python que puedas tener instaladas en tu sistema operativo.

2.1 ¿Cuáles son los beneficios de trabajar con entornos virtuales en proyectos de Python en Linux?

Permíteme destacar algunos puntos clave:

1. **Aislamiento:** Los entornos virtuales te permiten tener control total sobre las versiones de las bibliotecas y paquetes que utilizas en tus proyectos. Esto significa que puedes crear un ambiente aislado para cada proyecto, evitando conflictos y problemas de compatibilidad entre diferentes versiones de Python y sus dependencias.
2. **Portabilidad:** Al utilizar entornos virtuales, puedes compartir fácilmente tus proyectos con otros desarrolladores o ejecutarlos en diferentes máquinas sin preocuparte por las diferencias en las configuraciones del sistema. Todo lo que necesitas es compartir el archivo de requisitos del entorno virtual y cualquiera podrá recrear el mismo ambiente de trabajo en su propia máquina.
3. **Mantenimiento sencillo:** Los entornos virtuales facilitan la gestión de tus proyectos. Puedes instalar y actualizar paquetes de forma independiente dentro de cada ambiente virtual, sin afectar a otros proyectos o al sistema operativo en sí. Además, si algo sale mal en un entorno virtual, puedes solucionarlo sin que afecte a tus otros proyectos.
4. **Experimentación segura:** Si quieres probar una nueva biblioteca o una versión diferente de una dependencia en particular, un entorno virtual te proporciona un espacio seguro para hacerlo. Puedes instalar y probar nuevas

bibliotecas sin preocuparte de que afecten a otros proyectos o rompan la funcionalidad existente.

3 Anaconda

¿Qué es Anaconda? Es mucho más que una simple herramienta, es una plataforma completa de gestión de paquetes y entornos virtuales diseñada específicamente para los amantes de Python como tú.

Entonces, ¿por qué deberías elegir Anaconda frente a otras herramientas? Permíteme contarte algunas de las ventajas que hacen de Anaconda una elección excepcional:

1. **Gestión de paquetes simplificada:** Con Anaconda, olvídate de preocuparte por instalar y gestionar paquetes individualmente. Anaconda tiene su propio sistema de gestión de paquetes llamado “conda”, que te permite instalar, actualizar y eliminar paquetes de manera sencilla y eficiente. Además, Anaconda cuenta con un amplio repositorio de paquetes precompilados que puedes explorar y utilizar en tus proyectos con un solo comando.
2. **Creación de entornos virtuales sin complicaciones:** ¿Recuerdas la importancia de los entornos virtuales que discutimos anteriormente? Bueno, con Anaconda, crear y gestionar entornos virtuales es pan comido. Puedes crear un entorno virtual aislado para cada proyecto en cuestión de segundos. Además, puedes especificar fácilmente la versión de Python y las dependencias requeridas para cada entorno virtual, manteniendo todo organizado y libre de conflictos.
3. **Multiplataforma:** Ya sea que estés utilizando Linux, Windows o macOS, Anaconda te tiene cubierto. Esta plataforma es compatible con múltiples sistemas operativos, lo que significa que puedes disfrutar de las mismas

ventajas y características sin importar el entorno en el que te encuentres. Así que no importa si eres un entusiasta de Linux, un defensor de Windows o un fanático de macOS, Anaconda estará a tu lado.

4. **Integración con herramientas adicionales:** Anaconda no solo se detiene en la gestión de paquetes y entornos virtuales, sino que también ofrece una amplia gama de herramientas y utilidades adicionales que pueden mejorar tu flujo de trabajo. Desde el entorno de desarrollo integrado (IDE) llamado Anaconda Navigator, hasta la potencia de Jupyter Notebooks y la facilidad de distribución con Anaconda Cloud, hay muchas herramientas a tu disposición para mejorar tu productividad y simplificar tu desarrollo.

4 Instalación de Anaconda en Ubuntu Linux

Si estás utilizando Ubuntu Linux y te emociona comenzar a trabajar con Anaconda. Te guiaré a través de los pasos para descargar e instalar Anaconda en tu sistema Ubuntu Linux.

Paso 1: Descargar el instalador de Anaconda

Para empezar, debes visitar el sitio web oficial de Anaconda [Descargar Anaconda](#) y descargar el instalador adecuado para tu versión de Ubuntu Linux. Asegúrate de seleccionar la versión de Python que deseas utilizar y si tu sistema operativo es de 32 o 64 bits.

Paso 2: Abrir la terminal

Una vez que hayas descargado el instalador de Anaconda, abre la terminal en tu sistema Ubuntu Linux. Puedes hacerlo utilizando el atajo de teclado Ctrl + Alt + T o buscando “Terminal” en el menú de aplicaciones.

Paso 3: Navegar a la ubicación del instalador

En la terminal, navega hasta la ubicación donde descargaste el instalador de Anaconda. Por ejemplo, si lo descargaste en la carpeta “Descargas”, puedes usar el comando siguiente para ir a esa ubicación:

```
cd Descargas
```

Recuerda reemplazar “Descargas” con la carpeta en la que hayas guardado el archivo.

Paso 4: Ejecutar el instalador de Anaconda

Una vez que estés en la ubicación del instalador de Anaconda, puedes ejecutar el siguiente comando para iniciar el proceso de instalación:

```
bash nombre_del_instalador.sh
```

Asegúrate de reemplazar “nombre_del_instalador.sh” con el nombre real del archivo que descargaste.

Paso 5: Sigue las instrucciones de instalación

Después de ejecutar el comando anterior, seguirás las instrucciones del instalador de Anaconda en la terminal. Acepta los términos de licencia, selecciona la ubicación de instalación y responde cualquier pregunta adicional que se te presente durante el proceso de instalación.

Paso 6: Añadir Anaconda al PATH del sistema

Una vez que la instalación se complete, se te preguntará si deseas agregar Anaconda al PATH del sistema. Es recomendable seleccionar “yes” para que puedas acceder a los comandos de Anaconda desde cualquier ubicación en la terminal.

5 Configuración de un entorno virtual en Conda

Paso 1: Actualiza Conda:

Antes de empezar, es una buena práctica asegurarte de tener la versión más reciente de Conda. Abre tu terminal y ejecuta los siguientes comandos:

```
conda update conda
conda update --all
```

Esto actualizará Conda y todos los paquetes asociados.

Paso 2: Configura el entorno virtual:

Ahora, vamos a crear un nuevo entorno virtual. En tu terminal, ejecuta el siguiente comando:

```
conda create --name nombre_del_entorno
```

Reemplaza “nombre_del_entorno” con el nombre que desees darle a tu entorno virtual.

Paso 3: Activa el entorno virtual:

Una vez que hayas creado el entorno virtual, puedes activarlo con el siguiente comando:

```
conda activate nombre_del_entorno
```

Esto te permitirá trabajar en el entorno virtual específico.

Paso 4: Desactiva el entorno virtual:

Cuando hayas terminado de trabajar en tu entorno virtual y desees volver al entorno base, simplemente ejecuta el siguiente comando:

```
conda deactivate
```

Esto te llevará de vuelta al entorno base de Conda.

Paso 5: Elimina el entorno virtual:

Si en algún momento deseas eliminar un entorno virtual, ejecuta el siguiente comando:

```
conda env remove --name nombre_del_entorno
```

Asegúrate de reemplazar “nombre_del_entorno” con el nombre real del entorno que deseas eliminar.

Paso 6: Cambiar entorno

1. Abre Anaconda Navigator o el Anaconda Prompt (puedes encontrarlo en el menú de inicio de tu sistema operativo).
2. Una vez que hayas abierto el entorno de Anaconda, puedes verificar los entornos disponibles ejecutando el siguiente comando en el Anaconda Prompt:

```
conda env list
```

Esto te mostrará una lista de todos los entornos existentes.

3. Para cambiar al entorno de aprendizaje (llamado “learn” en este caso), utiliza el siguiente comando:

```
conda activate learn
```

Esto activará el entorno de aprendizaje y te permitirá trabajar en él.

4. Una vez en el entorno de aprendizaje, es posible que notes que no tiene instalados otros paquetes, aparte de los paquetes oficiales que vienen con Python. Si deseas tener un entorno relativamente limpio, puedes seguir estos pasos:

- Ejecuta el siguiente comando para abrir el intérprete de Python:

```
python
```

- Una vez dentro del intérprete de Python, ingresa el siguiente comando para importar el paquete “requests”:

```
import requests
```

Verás que se muestra un mensaje indicando que no se puede encontrar el paquete “requests”, lo cual es normal.

- Para salir del intérprete de Python, simplemente ingresa el siguiente comando:

```
exit()
```

Con esto, saldrás del intérprete de Python y volverás al Anaconda Prompt.

¡Y eso es todo! Ahora tienes los pasos detallados para configurar y administrar entornos virtuales en Conda.

6 Instalación de paquetes y bibliotecas en un entorno virtual

Cuando trabajas en proyectos de Python, es esencial tener acceso a las herramientas y funcionalidades adecuadas.

6.1 Uso de conda install

1. Asegúrate de tener tu entorno virtual activado. Si aún no lo has hecho, consulta el artículo anterior para aprender cómo activar tu entorno virtual específico.
2. Abre tu terminal o línea de comandos y ejecuta el siguiente comando para instalar un paquete desde el repositorio de Anaconda:

```
conda install nombre_del_paquete
```

Asegúrate de reemplazar “nombre_del_paquete” con el nombre real del paquete que deseas instalar.

3. Conda buscará el paquete en el repositorio de Anaconda y gestionará las dependencias automáticamente. Sigue las instrucciones en la terminal para confirmar la instalación.

6.2 Uso de pip install

1. Al igual que antes, asegúrate de tener tu entorno virtual activado.
2. Ejecuta el siguiente comando en tu terminal para instalar un paquete desde el Python Package Index (PyPI):

```
pip install nombre_del_paquete
```

Asegúrate de reemplazar “nombre_del_paquete” con el nombre real del paquete que deseas instalar.

3. Pip descargará el paquete desde PyPI y lo instalará en tu entorno virtual. Si el paquete tiene dependencias, pip también se encargará de resolverlas.

Conda es especialmente útil para instalar paquetes que son parte del repositorio de Anaconda, mientras que pip es más adecuado para paquetes que se encuentran en PyPI. Ambas herramientas son poderosas y te permiten acceder a una amplia gama de paquetes y bibliotecas para tus proyectos.

6.3 Ver información del paquete de entorno:

Para ver todos los paquetes instalados en el entorno actual, puedes utilizar el siguiente comando:

```
conda list
```

Al ejecutar este comando en el Anaconda Prompt, se mostrará una lista de todos los paquetes instalados en el entorno activo. Esto te permitirá conocer los paquetes y sus respectivas versiones que están disponibles en ese entorno.

6.4 Importar y exportar entornos:

Si deseas exportar la información del paquete del entorno actual, puedes utilizar el siguiente comando:

```
conda env export > environment.yaml
```

Este comando guarda la información del paquete en un archivo YAML llamado “environment.yaml”. El archivo contendrá la lista de paquetes y sus versiones que están instalados en el entorno actual.

Esta funcionalidad es útil cuando necesitas recrear el mismo entorno virtual en otro lugar. Para crear un nuevo entorno virtual utilizando el archivo de configuración, puedes utilizar el siguiente comando:

```
conda env create -f environment.yaml
```

Este comando creará un nuevo entorno virtual utilizando el archivo de configuración “environment.yaml”. El nuevo entorno tendrá los mismos paquetes y versiones que el entorno original, lo que facilita la replicación del mismo entorno en diferentes sistemas.

Estos pasos son útiles para compartir y recrear entornos virtuales con la misma configuración, lo que asegura que todos los paquetes necesarios estén disponibles.

7 Conclusión

En conclusión, el uso de Anaconda se presenta como una solución elegante y sencilla para abordar las desventajas de entorno de Python. A través de Anaconda, se puede gestionar de manera eficiente la instalación y actualización de paquetes, así como la creación y exportación de entornos virtuales. Sin embargo, es importante destacar que la implementación de estas funcionalidades no es mágica, requiere comprensión y familiaridad con los comandos y procesos asociados.

Además de la gestión de paquetes, Anaconda ofrece una amplia gama de herramientas y paquetes para el análisis de datos, lo cual constituye otro aspecto valioso de su funcionalidad. Sin embargo, en este contexto, nos hemos enfocado en aprender cómo utilizar Anaconda para cambiar el entorno de desarrollo de manera efectiva, lo cual ha representado una mejora significativa en comparación con el enfoque tradicional.

¡Happy coding!

8 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. [!\[\]\(5991ec24bc84372ce6cc8dfe116dafcc_img.jpg\) Instalacion De Anaconda](#)
2. [!\[\]\(7d1cc7a4e7c5677a8c7622e62b7cbd13_img.jpg\) Configurar Entorno Virtual Python Anaconda](#)
3. [!\[\]\(7b39db77b629a97dd1b3a010bdb8ffac_img.jpg\) 01 Introducion A La Programacion Con Python](#)
4. [!\[\]\(4c64f1aa40817a3ed944fd7082d06782_img.jpg\) 02 Variables Expresiones Y Statements Con Python](#)
5. [!\[\]\(dd014b3c3914024600e3d36b08949622_img.jpg\) 03 Objetos De Python](#)
6. [!\[\]\(bbf5fe46837b7857abcd77effd234ea4_img.jpg\) 04 Ejecucion Condicional Con Python](#)
7. [!\[\]\(e2150d94656ac55bc5e6e01dbb3ebbc5_img.jpg\) 05 Iteraciones Con Python](#)

8.  [06 Funciones Con Python](#)
9.  [07 Dataframes Con Python](#)
10.  [08 Prediccion Y Metrica De Performance Con Python](#)
11.  [09 Metodos De Machine Learning Para Clasificacion Con Python](#)
12.  [10 Metodos De Machine Learning Para Regresion Con Python](#)
13.  [11 Validacion Cruzada Y Composicion Del Modelo Con Python](#)
14.  [Visualizacion De Datos Con Python](#)

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!